SIH26171

Final Technical Execution Plan

On-device Visual Perception for Light-weight Browser Agents — ISRO. Consolidated: own-built versioned memory, foveated vision, numbered-tag grounding, deterministic multi-action execution, draft-model speculative planning, 3-language voice, and Proof-of-Perception — pure engineering, no non-technical filler.

Track	Sponsor	Team	Window	Phases
Software	ISRO	4 members	6 days	162

Key decisions locked in: memory is self-built (Chroma + versioning), not Mem0 or OpenClaw — fully your own code, easier to defend, and avoids known reliability gaps in library-based memory. Voice supports Hindi, Kannada, and English via local AI4Bharat/Indic-tuned Whisper models. Model testing and hardware-dependent tuning is owned by Siddu and Mohit, since they have the strongest laptops.

Team & Roles

Siddu — Integration Lead — Memory, Vision & Foveation
Owns integration, the self-built versioned memory system, the full vision/foveation pipeline, numbered-tag grounding, Proof-of-Perception evidence, and the hash-chain audit log.

Mohit — Extension, DOM Filtering & Multi-Action UI
Owns the extension itself, the semantic DOM-filtering content script, cropping/overlay rendering, the native multi-action executor, and all UI including the evidence panel.

Omkar — Agent Reasoning, Draft Model & Voice
Owns the native host bridge, the agent loop, the draft-model speculative planner, guardrail validation, and local voice transcription across Hindi, Kannada, and English.

Chinmay — Independent Track — Dashboard, QA, Benchmarking & Security (night shift)
Owns the mock demo dashboard, encryption, adversarial and QA testing, and the speed/accuracy benchmark harness — fully self-contained, handed off each morning.

Day 0 — Research & Architecture Lock

Siddu

1. Repo + integration branch strategy
Create the shared repo layout (extension/, native-host/, memory/, vision/) and a branch rule where only Siddu merges into integration. Done when everyone's pushed a hello-world commit.

2. Versioned memory schema design
Design 4 Chroma collections (session_memory, user_preferences, site_knowledge, task_history) with a version/superseded field so updated facts replace old ones instead of both surviving — this is the exact weakness that hurts library-based memory tools. Done when schema is written up and reviewed.

3. Download and study the official ISRO dataset
Get the dataset for SIH26171 and understand what 'visual perception' concretely means for this data. Done when you can describe it in one paragraph.

4. Master architecture diagram
Draw the full flow including the new speed layers: Extension ↔ DOM-filter ↔ Native Host ↔ Draft Model ↔ Vision (foveated) ↔ Memory ↔ back to Extension. Done when the team confirms they understand where their piece fits.

5. Extension↔host message contract
Define the JSON contract for every message type, now including multi-action plans and cropped-patch payloads. Done when Mohit and Omkar sign off.

6. Chroma persistent local-client setup
Init a PersistentClient on a local folder, confirm offline read/write with wifi off. Done when a write→restart→read cycle works with zero network.

Mohit

7. Extension project skeleton
Manifest V3, popup.html, background worker, content script — confirm it loads unpacked with zero errors. Done when a blank popup shows.

8. Permissions research
Identify exact permissions needed (activeTab, scripting, storage, tabs, offscreen). Done when list is ready for the manifest.

9. UI wireframe including numbered-tag overlay + evidence panel
Sketch popup/sidebar layout: command input, mic button, reasoning-trail with evidence crops, numbered-tag toggle. Done when Siddu approves the layout.

10. Dev reload workflow
Set up fast build/reload so code changes are visible in under 10 seconds. Done when this works.

Omkar

11. Native-messaging-host registration research
Understand OS-level registration for the native host before building against it. Done when you can explain the steps.

12. Pull and shortlist vision + text models WITH Siddu and Mohit
Since Siddu and Mohit have the strongest hardware, all three of you jointly pull and test candidate models (moondream/qwen2-vl-2b for vision, qwen2.5-3b/llama3.2-3b for text, plus a ~0.5B draft model). Omkar owns writing up the tool-calling reliability results. Done when a comparison table exists.

13. Native host skeleton
Standalone Python/Node project that runs independently for testing. Done when it prints a startup message.

14. Agent-loop pattern research
Study plan→act→verify patterns (LangGraph-style) and sketch how multi-action plans fit in. Done when you can describe the loop without notes.

Chinmay

15. Mock ISRO-style dashboard skeleton
Start a standalone local web project — login page, main page shell — fully independent, no dependency on anyone else's code tonight. Done when localhost serves a running page.

16. Study the ISRO dataset structure independently
Once Siddu shares it, review it on your own and write a short summary. Done when you can describe its format in your own words.

17. Local encryption research
Research SQLCipher/file-level encryption for the memory DB. Done when you have a working code snippet ready to hand off.

Day 1 — Foundations — Parallel Build

Siddu

18. Screenshot capture + preprocessing pipeline
Crop/resize/normalize captured screenshots into a reusable function. Done when any screenshot converts cleanly.

19. Interactive-region locator (foveation, part 1)
From the filtered DOM/JSON, identify bounding boxes of the currently relevant interactive sub-regions — this feeds the cropping step Mohit builds next. Done when it correctly finds the 2–3 most relevant regions on a test page.

20. Numbered-tag grounding system
Build the set-of-marks generator: assign each interactive element a temporary ID and render a numbered tag next to it, so the model only ever outputs a number, never coordinates. Done when a test image shows correct numbered overlays.

21. Embedding wrapper + versioned store_memory()
Text-to-vector wrapper via local Ollama embeddings, plus a memory-write function that marks old facts superseded rather than duplicating them. Done when a round-trip write/update/retrieve test shows only the latest version returned.

22. Semantic retrieve_memory() across collections
Query function returning top-k relevant entries per collection. Done when a paraphrased test query returns the right stored fact.

Mohit

23. Manifest + real permissions
Fill in final manifest with permissions, content-script registration. Done when Chrome shows zero warnings.

24. Semantic DOM-filtering content script
Strip script/style/hidden elements, keep only interactive elements + labels — target ~90% payload reduction vs raw HTML. Build this as a two-pass process: first pass removes non-visible and non-interactive nodes entirely (display:none, zero-dimension elements, tracking pixels), second pass extracts only the semantic attributes the model actually needs (tag type, visible text, ARIA label, rough position) rather than full computed styles. Avoid the trap of just minifying HTML text — the goal is structural compression, not string compression, since a shorter but still HTML-shaped payload still costs unnecessary tokens. Done when tested against 3 different page types and payload size reduction is measured, not assumed.

25. Compress filtered output into minified JSON tree
Convert filtered DOM into a compact structural JSON matching Siddu's overlay/memory format. Done when JSON is measurably smaller than filtered HTML.

26. Basic screenshot capture wiring
Wire chrome.tabs.captureVisibleTab and confirm a usable image is returned. Done when a real tab capture displays correctly.

27. Popup UI v1
Input box, send button, response area — functional, not styled. Done when typing and sending logs correctly.

Omkar

28. Native-host skeleton + ping-pong messaging
Register with Chrome, confirm a round-trip test message works. Done when this is reliable.

29. Ollama client wrapper
Send prompt → receive response, basic error handling. Done when it gets a real model response.

30. Structured JSON output enforcement
Pydantic schema validation on every model response; reject malformed output. Done when a bad response is correctly caught.

31. Draft-model wrapper (speculative planning)
Wire the tiny ~0.5B text model as a fast first-pass planner against filtered DOM/JSON. This mirrors speculative decoding from LLM inference literature: a cheap, fast model proposes a plan, and it's only escalated to a slower, more capable model when needed — the win comes from the cheap model being right often enough that the average cost per action drops significantly, even though any single ambiguous case costs more (draft attempt + full escalation). Keep the draft model's prompt minimal — just the filtered element list and task — since its whole value proposition is speed, and a bloated prompt defeats that. Done when it proposes a plausible action plan on a clear test page, and you've measured its average response time against the full text model's.

32. Multi-action plan schema
Define the JSON shape for a plan (ordered list of actions) instead of one action at a time. Done when schema is agreed with Mohit.

Chinmay

33. Mock dashboard login page
Fake login form, no real auth needed. Done when it renders and navigates forward.

34. Mock dashboard main page + data table
Realistic-looking table with 10–15 rows. Done when it renders cleanly.

35. Canvas-based widget
One deliberately non-DOM-friendly element to force vision fallback testing later. Done when it's clearly not readable via DOM text.

36. Confirm dashboard runs fully offline
No external CDN/script dependency. Done when it works with wifi off.

Day 2 — Core Capability Build

Siddu

37. Full vision pipeline assembly
Connect capture → crop (foveation) → numbered overlay → model call → selection into one function, with each stage's output feeding cleanly into the next via well-defined data structures rather than loosely-typed dicts passed around ad hoc. This is the backbone every later optimization (parallelization, caching, prefetch) attaches to, so get the interfaces right now rather than patching them later. Log timing at each internal stage from the start, not added as an afterthought, since you'll need this data for Day 4's benchmarking. Done when it works start-to-finish on a real page and each stage's individual timing is visible in logs.

38. Offscreen canvas cropping integration
Work with Mohit's cropped patches as vision-pipeline input instead of full screenshots. Done when cropped-patch inference is measurably faster than full-frame.

39. Guardrail validation design
Define which action types (submit/delete/confirm) require cross-checking the chosen element's real label against stated intent before executing. Done when the rule is written and testable.

40. DOM-vs-vision decision router
Route to vision only when DOM/JSON data is missing or ambiguous. Done when it correctly skips vision on DOM-friendly pages and triggers it on the canvas widget.

41. Memory versioning stress test
Deliberately overwrite the same fact 3–4 times and confirm only the latest version is ever retrieved. Done when old versions never leak into results.

Mohit

42. Offscreen canvas cropping implementation
Use an offscreen canvas to crop screenshots to Siddu's located regions — small patches instead of full frames. Done when patches visually match intended regions.

43. Numbered-tag overlay renderer
Actually render numbered boxes on the live page from Siddu's grounding data. Done when boxes appear correctly over real elements.

44. Native multi-action executor
Execute an entire returned action plan against the DOM in sequence, without a model call between steps. Implement this with a per-step existence check immediately before each individual action fires (not just once at the start of the plan), since a page can change mid-plan even within a few hundred milliseconds — if step 3's target element vanished because step 2's action changed the page, abort the remaining plan and fall back to fresh single-step reasoning rather than blindly continuing. Log which steps executed successfully before any failure, so the verification/retry logic (Omkar's) knows exactly where to resume from. Done when a 3-step plan runs fully with zero intermediate model calls, and a deliberately broken step 2 correctly halts steps 3+.

45. Background-script relay logic
Relay DOM/screenshot/plan data between content script and native host. Done when a full round trip works with dummy data.

Omkar

46. DOM-based reasoning prompt (fast default path)
Prompt template giving the model a structured element list + task, expecting a plan or single action. Done when it correctly selects on 4–5 test cases.

47. Draft-model escalation trigger
Draft model hands off to the full vision pipeline only on ambiguous/unmapped layouts. Done when ambiguous cases escalate and clear ones don't.

48. Verification loop (per-plan, not per-step)
Verify end state once after a full multi-action plan executes; fall back to single-step + re-verify if it fails. Done when a broken plan correctly triggers fallback.

49. Model warm-keeping + quantization benchmark
Keep models loaded between calls; benchmark 4-bit quantized vs full precision on real hardware. Done when concrete speed numbers exist.

Chinmay

50. Second dashboard page (navigation target)
Linked reports/details page. Done when navigation works.

51. Activity/audit-log schema design
Timestamp, action, target, result, plus a hash-of-previous-entry field for tamper-evidence. Done when schema is documented.

52. Begin independent nightly QA on merged code
Pull latest integration branch, test independently, log bugs with repro steps. Done when first bug batch is logged with owners assigned.

Day 3 — Full Integration + Voice

Siddu

53. Integrate real extension output into vision/memory pipeline
Replace mocked DOM/screenshot input with Mohit's live data. Done when a real page flows correctly through overlay/router.

54. Wire live memory retrieval into the agent loop
Real Chroma queries replace any mocked calls. Done when a real command retrieves sensible context.

55. Proof-of-Perception evidence capture
For every action, capture the exact DOM element/vision crop that justified it, plus a one-line reason. Block actions with no evidence. Done when a real action produces a full evidence record and an evidence-less candidate is refused.

56. Why-panel explainability output
Output why an element was chosen/rejected, formatted for Mohit's UI panel. Done when a real decision produces a readable explanation.

57. Confidence-gated pause-and-confirm
Below-threshold confidence blocks the action and asks for confirmation instead of guessing. Done when an ambiguous test case is blocked every time.

58. Site_knowledge auto-population
First successful visit to a page writes a layout summary to memory for faster future handling. Done when a second visit visibly uses cached knowledge.

Mohit

59. Wire live native-host communication
Real command flow through popup → background → host → back. Done when a real command reaches the host and returns a real response.

60. Mic button + audio recording
Capture audio, send to native host. Done when a recorded clip is sent successfully.

61. Evidence panel UI
Show the crop/DOM snippet next to each reasoning-trail step. Done when real evidence renders during a live run, not placeholders.

62. Confidence + refusal UI state
Distinct, calm UI for low-confidence pauses — not an error box. Done when this state is tested and polished.

Omkar

63. Voice transcription — Hindi, Kannada, English
Integrate an AI4Bharat IndicConformer or Indic-tuned Whisper checkpoint per language (3 checkpoints total), running locally via transformers/Whisper.cpp equivalent. Done when a spoken command in each of the 3 languages is correctly transcribed offline.

64. Route transcribed text into the existing command pipeline
Whichever language is spoken, output normalizes into the same downstream command flow. Done when all 3 languages successfully trigger a real action end to end.

65. Guardrail validation — live implementation
Cross-check chosen element's real label against stated intent before submit/delete/confirm actions; intercept mismatches. Done when a deliberately mismatched case is caught.

66. Wire memory-aware + evidence-aware prompt construction
Prompts now include retrieved memory AND require evidence before an action is finalized. Done when a real prompt visibly contains both.

Chinmay

67. Independent QA on dynamic content + offline scenario
Test edge cases and the full ‘pull the plug’ scenario against the latest merged code, nightly, without needing anyone live. Done when results are logged with a clear pass/fail.

68. Implement local encryption for the memory DB
Self-contained SQLCipher module others can drop in. Done when a memory DB file is demonstrably unreadable without the key.

69. Verify audit-log entries are complete

Check real logs from the merged code against your schema. Done when a full task run's logs are confirmed accurate.

Day 4 — Differentiators, Speed & Hardening

Siddu

70. Full graceful-degradation ladder
DOM → numbered-tag vision (cropped) → full-page vision → explained failure, as one chain. Done when all 4 tiers are triggered deliberately at least once.

71. Hash-chain audit log implementation
Each log entry includes a hash of the previous entry — tamper-evident, fully local. Done when editing a past entry is detectably broken on re-verification.

72. Workflow-schema caching
Extend site_knowledge: first successful multi-step flow gets cached; repeat runs skip the model entirely and execute deterministically. Done when a second run is visibly faster with zero model calls.

73. Stress-test vision against the real ISRO dataset
Run the pipeline against actual dataset samples, not just your own test pages. Done when you have real accuracy numbers.

74. Full offline test
Disconnect network entirely, run the complete demo task. Done when it completes with zero connectivity.

Mohit

75. One-click log-verification button
Re-walks the hash chain and reports pass/fail, as a live demo proof point. Done when it correctly passes on an untouched log and fails on a tampered one.

76. Move screenshot capture off the main thread
Web Worker/OffscreenCanvas so capture doesn't stutter the page. Done when no visible frame drops occur.

77. Resource-usage dashboard
Live RAM, per-action inference time, model size, fed from the native host. Done when numbers match a real system monitor.

78. Cache-invalidation UI feedback
Show clearly when a cached workflow is being replayed vs. freshly reasoned. Done when this is visible during a live run.

Omkar

79. Cache correctness on changed pages
Confirm a cached workflow correctly detects a changed layout and falls back to fresh reasoning instead of blindly replaying. Done when a deliberately altered page triggers cache invalidation.

80. Draft-model latency measurement
Compare average per-action latency with vs. without the draft-model shortcut, on your own hardware. Done when you have real numbers, not assumed percentages.

81. Structured decision logging
Every agent decision point logged in a structured format feeding the audit trail. Done when a full run produces a complete decision log.

82. Lock final model choices + quantization
Based on all benchmark data, freeze vision/text/draft/voice model choices and quantization levels. Done when this is decided and documented, not revisited again.

Chinmay

83. Adversarial hallucination test list
Write 8–10 commands designed to tempt the model into inventing elements/facts — missing elements, ambiguous intent, no-memory-match cases. Done when the list covers at least 3 failure categories.

84. Run adversarial list against the current build
Confirm every case correctly refuses or asks for clarification rather than acting wrongly. Done when a full pass shows zero false-confident actions.

85. Full regression QA pass
Re-test every feature built so far, not just today's additions. Done when you have an up-to-date pass/fail list.

86. Build the speed/accuracy benchmark harness

Script that runs the same test tasks with each optimization toggled on/off, logging timing and success rate. Done when it produces a real before/after table for at least 3 optimizations.

Day 5 — Polish, Verification & Rehearsal

Siddu

87. Final integration pass + code freeze
Merge final fixes, freeze the demo codebase. Done when the team agrees this exact version is what gets demoed.

88. Verify the memory demo moment
Test the paraphrased-repeat-command retrieval multiple times. Done when it succeeds at least 4 of 5 attempts.

89. Verify Proof-of-Perception + hash-chain demo moments
Run the evidence-capture and log-verification moments repeatedly for reliability. Done when both work consistently on demand.

90. Full timed dry-run demo
Run the entire demo script with the team, timed. Done when it completes within your allotted slot.

91. Architecture Q&A; prep
Plain-language answers on memory, vision, grounding, and why-not-Mem0/OpenClaw. Done when you can answer without hesitation.

Mohit

92. Final UI polish pass
Typography, spacing, visual consistency across the whole extension. Done when nothing looks unfinished.

93. Clean-install flow test
Fresh Chrome profile, under 15 seconds to first working command. Done when this works flawlessly.

94. Rehearse the UI-facing demo portion
Practice narrating and operating the UI live. Done when you can run it without notes.

Omkar

95. Final reliability pass on the agent loop
No ungraceful crashes; every error path shows a clear message. Done when every known error scenario is tested clean.

96. Rehearse triggering fallback scenarios live
Practice forcing low-confidence, vision-fallback, and offline moments on demand. Done when you can trigger each reliably.

97. Voice demo rehearsal — all 3 languages
Practice the live voice-command moment in Hindi, Kannada, and English. Done when all 3 work reliably in rehearsal.

Chinmay

98. Final QA sign-off
Confirm every planned demo moment works on the final frozen code. Done when you can confidently say yes to every part of the script.

99. Finalize real benchmark numbers for the pitch
Replace any borrowed/estimated stats with your harness's actual measured results. Done when every quoted number in the pitch is your own.

100. Full team rehearsal, timed
Run the complete demo with everyone present. Done when it completes cleanly within time.

Advanced Speed Engineering — Pipeline-Level Optimizations

New this round: parallelizing independent pipeline stages, incremental DOM diffing, KV-cache reuse, predictive prefetch, and operator-level model optimization — real inference-pipeline engineering, not UI polish. Attempt after the corresponding Day 0–4 core phases for that stage are solid.

Siddu

101. Parallelize memory retrieval with vision/DOM analysis

Right now memory retrieval and page-perception likely run one after another inside a single request — change this so both start at the same time as soon as a command comes in, using asyncio (or Promise.all on the JS side) instead of sequential await calls. Memory retrieval only needs the command text, not the page state, so there's no real dependency forcing them to be sequential. On a task where memory lookup takes ~150ms and vision analysis takes ~400ms, running them in parallel instead of back-to-back saves roughly that 150ms on every single action, which compounds fast across a multi-step task. Done when a profiler/log timestamp shows both operations starting within the same few milliseconds of each other, not one waiting for the other to finish first.

102. Batch embedding computation for memory writes

When multiple facts get written to memory in quick succession (e.g. after a multi-step task completes), don't call the embedding model once per fact — batch them into a single embedding call, since most local embedding models support batched input natively and batching amortizes the fixed per-call overhead across multiple items. Group memory writes that happen within the same task-completion event into one batched embed-then-store operation instead of N separate ones. Done when writing 5 facts at once is measurably faster than 5 sequential single-fact writes on your own hardware, not just assumed to be faster.

103. Operator-level optimization pass on the vision model

Once your final vision model and quantization level are locked, run it through ONNX Runtime's graph optimizer (or Ollama's built-in equivalent if using GGUF) to fuse compatible operators and eliminate redundant computation graph nodes — this is a free speed gain that requires no accuracy trade-off, unlike quantization. Compare inference time before and after optimization on the same test image set to confirm the gain is real on your specific hardware, since optimizer effectiveness varies by CPU/GPU architecture. Done when you have a measured before/after inference-time comparison specifically isolating the optimization pass's effect.

Mohit

104. Incremental DOM diffing instead of full re-extraction

Currently every reasoning cycle likely re-parses the entire page's DOM from scratch, even when only a small part of the page changed since the last read (e.g. one field got filled in). Implement a MutationObserver-based diffing layer that tracks which specific DOM nodes changed since the last extraction, and only re-runs the semantic filter/JSON-tree builder on the changed subtree, merging it into the existing cached tree rather than rebuilding the whole thing. This matters most on data-dense pages where full re-extraction is expensive but the actual per-step change is tiny. Done when a test page shows extraction time scaling with the size of the change, not the size of the whole page.

105. Predictive prefetch of the next likely page state

While the current action is executing (e.g. a form submit that will trigger navigation), speculatively start DOM/screenshot capture of the anticipated next page state as soon as the action fires, rather than waiting for the action to fully complete before beginning the next extraction cycle. This overlaps network/render wait time with extraction work instead of doing them strictly sequentially. Build this carefully with a cancellation/invalidation path in case the actual resulting page differs from what was predicted — a wrong prefetch should be cheaply discarded, not block the real extraction. Done when a navigation-triggering action shows measurably less total latency to the next ready state versus the non-prefetched version.

106. Web Worker offload for JSON tree construction

Move the DOM-to-JSON compression step (from Day 1) into a dedicated Web Worker so it doesn't compete with the main thread for CPU time while the page is also rendering the numbered-tag overlay. This keeps the visible UI responsive during heavier extraction work on data-dense pages. Done when extraction work no longer causes any visible UI jank during a live test on a complex page.

Omkar

107. Reuse KV-cache across consecutive reasoning calls within one task

When your agent makes several reasoning calls in a row for the same multi-step task, a meaningful part of the prompt (system instructions, task description, memory context) often stays identical between calls — configure Ollama/your inference server to keep and reuse the key-value cache for that shared prefix instead of recomputing it from scratch on every call. This can meaningfully cut time-to-first-token on the second and later calls of a task, since only the new/changed part of the prompt needs fresh computation. Done when consecutive same-task calls show lower latency than the first cold call, beyond what warm-keeping alone explains.

108. Stream partial output and act on safe prefixes early

For actions where the model's output structure allows it (e.g. a plan is a list and the first item is often complete before the full list finishes generating), stream tokens and begin validating/preparing the first action as soon as it's syntactically complete, rather than waiting for the entire response to finish generating before any downstream work starts. This is a genuine latency-hiding technique, not a shortcut on correctness — the action still only executes after full validation. Done when a multi-action plan's first step begins validation visibly before the full plan has finished streaming.

109. Model warm-swap pipeline overlap

When the agent is likely to need a different model next (e.g. finishing a text-reasoning step and about to need the vision model for a canvas element), start warming/loading that next model in the background while the current step is still finishing, rather than loading it only after the need becomes certain. This trades a small amount of speculative wasted work (on the cases where the guess is wrong) for a real latency win on the cases where it's right. Done when a text-to-vision handoff shows reduced cold-start delay compared to loading vision only after the handoff is confirmed.

Chinmay

110. Build a per-phase latency breakdown, not just end-to-end timing

Extend the benchmark harness to log timing for each individual pipeline stage — DOM extraction, memory retrieval, model inference, action execution — separately, not just total task time, so the team can see exactly which stage benefits from which optimization rather than guessing. Done when a single task run produces a stage-by-stage timing breakdown, not just one total number.

111. Verify parallelized stages don't introduce race conditions

Independently stress-test the newly parallelized memory-retrieval/vision-analysis pipeline and the prefetch logic for race conditions — cases where a stale prefetch result gets used, or a memory query returns after it's no longer needed. Done when you've specifically tried to break the parallel/prefetch logic and logged whether it held up.

Advanced Architecture & Stretch (if time allows)

Genuine additional engineering, not filler — attempt only after a person's core Day 0–5 phases are done.

Siddu

112. WebGPU in-browser inference experiment (Transformers.js v3)
Test running a small ONNX model directly in the browser via WebGPU, bypassing the native host for lightweight steps — genuinely cutting-edge if it works. Done when a minimal in-extension WebGPU inference test succeeds, or you make an informed decision to skip it.

113. Memory-conflict resolution beyond basic versioning
Handle the case where two updates to the same fact arrive close together — keep most recent, log the override. Done when a deliberate conflict test resolves predictably.

114. Similarity-score threshold for memory injection
Never inject irrelevant memories below a similarity cutoff. Done when a clearly irrelevant query returns nothing.

115. Memory retrieval speed test at 500+ entries
Load synthetic entries and confirm retrieval stays demo-fast at scale. Done when timing is measured and acceptable.

116. Cross-check foveation accuracy against full-frame baseline
Compare cropped-patch vision accuracy against full-screenshot accuracy on the same test set, to confirm speed didn't cost correctness. Done when both are measured side by side.

117. Second vision model as high-accuracy fallback toggle
Optional larger model available for harder cases, clearly trading speed for accuracy. Done when the toggle works and the difference is measurable.

118. Ollama-load-failure graceful handling
Clear error state if Ollama isn't running, instead of a silent hang. Done when this is tested and handled.

119. Final code-cleanup pass on integration layer
Readability pass across the glue code others will read during Q&A.; Done when the integration code is presentable.

Mohit

120. Language-selector UI for voice input
Simple switch/auto-detect between Hindi, Kannada, and English so the mic knows which model to route to. Done when switching languages correctly changes transcription behavior.

121. Multi-action plan visualization
Show the full planned action sequence in the UI before/as it executes, not just one step at a time. Done when a 3-step plan displays clearly.

Omkar

122. Runtime model switching without restart
Switch between two loaded models (e.g. draft vs. full) without restarting the host. Done when switching works without a crash.

123. Health-check endpoint
UI can query ‘backend connected’ status. Done when it reflects real host state.

124. Request queueing for rapid repeated commands
Prevent overload from double-submission. Done when this is handled gracefully.

125. Language-detection fallback for voice
If no language is manually selected, attempt auto-detection across the 3 supported languages before transcribing. Done when this correctly routes at least 2 of 3 languages without manual selection.

126. Prompt-size optimization pass
Trim unnecessary verbosity from all prompt templates for faster inference. Done when token count is measurably reduced without accuracy loss.

127. Load-test the full agent loop 10+ consecutive runs
Confirm no crashes or memory leaks over repeated use. Done when 10 consecutive tasks complete cleanly.

128. Temperature/sampling tuning pass

Lock in the most consistent settings across text, vision, and draft models. Done when repeated identical commands produce consistent results.

Chinmay

129. Verify resource-usage dashboard numbers independently
Cross-check Siddu/Mohit's live dashboard against your own system monitor. Done when accuracy is independently confirmed.

130. Stress-test the demo dashboard for flakiness
Rapid repeated interaction testing on your own dashboard. Done when you're confident it won't be a point of failure.

131. Independent voice-command testing across all 3 languages
Test Hindi, Kannada, and English commands yourself against the merged build, nightly. Done when you've logged pass/fail for each language independently.

132. Verify workflow-caching correctness independently
Test cache hit/invalidation behavior yourself, separate from Siddu's own testing, as a second set of eyes. Done when you confirm or dispute the cache behaves correctly.

133. Final benchmark-harness cross-check
Re-run the harness one more time on the frozen build the night before, to catch any late regression. Done when final numbers match what's going into the pitch.

Verification & Technical Documentation

Siddu

134. Unit tests for the DOM-vs-vision router boundary
Confirm the router's decision logic is correct at the edge cases, not just obvious ones. Done when 5+ boundary cases pass.

135. Unit tests for memory versioning edge cases
Test rapid sequential updates to the same fact, and updates arriving out of order. Done when both cases resolve correctly.

136. Document the final prompt templates used
Write up the exact prompts for reasoning, vision selection, and guardrail checks — needed for technical Q&A; and the written submission. Done when this matches the actual frozen code.

137. Verify evidence records are complete for every action type
Check that click, type, and scroll actions all produce valid evidence, not just clicks. Done when all 3 action types are confirmed.

Mohit

138. Test DOM filtering against a JS-heavy dynamic page
Confirm the semantic filter still performs well when content loads asynchronously. Done when tested and any gaps are logged.

139. Test numbered-tag overlay at different zoom levels
Confirm overlay positioning stays accurate if the page is zoomed. Done when tested at 2–3 zoom levels.

140. Cross-check multi-action executor against single-action baseline
Confirm the batched executor produces identical results to doing the same steps one at a time. Done when both approaches agree on a test task.

141. Verify UI performance with Web Workers under load
Confirm no dropped frames during a full demo-length session, not just a quick test. Done when a full run shows no stutter.

Omkar

142. Test structured-output validation against malformed edge cases
Feed the schema validator deliberately broken JSON in several different ways. Done when every malformed case is caught.

143. Verify draft-model + full-vision handoff timing
Confirm the escalation from draft to full vision doesn't introduce a visible stall. Done when handoff timing is measured and acceptable.

144. Cross-check guardrail validation against non-obvious mismatches
Test cases where the label is subtly wrong (e.g. ‘Confirm’ vs ‘Confirm Cancel’), not just obviously wrong ones. Done when subtle cases are still caught.

145. Document the agent-loop and tool-calling architecture
Write up the plan→act→verify loop and multi-action schema for the technical report. Done when this matches the frozen code.

Chinmay

146. Independent test of the guardrail on the mock dashboard's real forms
Try to trigger a guardrail failure yourself using the actual demo dashboard, not synthetic cases. Done when you've logged real pass/fail results.

147. Independent verification of Proof-of-Perception evidence accuracy
Spot-check that captured evidence genuinely matches the action taken, across 5+ real runs. Done when you can confirm evidence is accurate, not just present.

148. Final adversarial + voice-language combined test pass
Run adversarial commands in all 3 supported languages, not just English, to confirm hallucination resistance holds across languages. Done when this combined test is logged.

Efficiency Micro-Optimizations

Siddu

149. Cache embedding vectors for repeated identical queries
Avoid re-embedding the exact same query text twice in a session. Done when a repeated identical command skips re-embedding.

150. Trim memory context to top-3 facts max per prompt
Even after retrieval, hard-cap what gets injected to keep prompts small and fast. Done when no prompt exceeds the cap.

151. Precompute and cache the ISRO-dataset benchmark results
Store your accuracy benchmark results so they don't need re-running before every rehearsal. Done when results are saved and reusable.

Mohit

152. Debounce rapid DOM-mutation events
Avoid re-triggering extraction on every tiny DOM change; batch them. Done when rapid page updates don't cause redundant extraction calls.

153. Lazy-load the evidence panel images
Only render evidence crops as they're needed, not all upfront. Done when this measurably improves UI responsiveness on a long session.

154. Minify and bundle the extension's final build
Reduce load time and file size for the install-in-10-seconds goal. Done when the packaged extension loads faster than the unminified dev version.

Omkar

155. Batch consecutive read-only DOM queries
Combine multiple lookups into fewer native-host round trips where possible. Done when round-trip count is measurably reduced for a multi-step task.

156. Preload the most-likely-next model based on task pattern
If a task is likely to need vision next, start warming it slightly earlier. Done when this reduces perceived latency in testing.

157. Cap max plan length per multi-action batch
Prevent overly long speculative plans from a single reasoning call; re-verify sooner instead. Done when plans longer than the cap are automatically split.

Chinmay

158. Time the full demo task under best-case vs worst-case conditions
Measure timing with all optimizations on vs. a baseline with them off, for the pitch's honest comparison slide-equivalent talking point. Done when both numbers are recorded.

Final Cross-Team Technical Checks

Siddu

159. Confirm zero network calls across the entire pipeline
Use a network monitor to verify literally nothing — memory, vision, reasoning, voice — makes an external call during a full task run. Done when a full run shows zero outbound network activity.

160. Reconcile all benchmark numbers into one final reference sheet
Pull every measured number (latency, accuracy, payload reduction) from across the week into one sheet the team quotes consistently. Done when this sheet is final and matches the frozen build.

Omkar

161. Confirm all 3 voice models and all text/vision models coexist in memory without exceeding hardware limits
Load everything the demo needs simultaneously and confirm no out-of-memory failures on the actual demo laptop. Done when a full session runs without memory errors.

Chinmay

162. Final independent full-system regression run
One last complete pass through every feature on the frozen build, purely independent verification the night before. Done when you can sign off with zero open blocking bugs.

Production-Quality Checklist

- ✓ Zero network calls anywhere in the pipeline — memory, vision, reasoning, voice all confirmed local
- ✓ Memory correctly resolves paraphrased repeat commands using only the latest version of any updated fact
- ✓ Numbered-tag grounding used for every action — the model never outputs a raw coordinate
- ✓ Multi-action plans execute without a model call between steps, verified once per plan
- ✓ Draft model correctly escalates to full vision only on ambiguous cases
- ✓ Guardrail validation catches at least one subtle (not just obvious) label mismatch live
- ✓ Voice works reliably in all 3 languages — Hindi, Kannada, English — tested independently by Chinmay
- ✓ Proof-of-Perception evidence is present and accurate for every action type, and the hash-chain log verification has been demonstrated both passing and correctly failing
- ✓ Workflow caching correctly invalidates on a changed page instead of blindly replaying
- ✓ Every speed/accuracy number in the pitch comes from Chinmay's benchmark harness, not an external source